

**ARQUITETURA DE SOFTWARE - CONCEITOS, PADRÕES, ESCOLHA DE
ARQUITETURA E TENDÊNCIAS**

ATIVIDADE 2

Prof.º Denilce de Almeida

Disciplina: Programação para WEB

Murilo Ferrari Menezes, 0030482511023

Sorocaba

Agosto/2026

Sumário

1. Conceitos.....	3
2. Padrões.....	4
2.1. Análise dos Principais Padrões e Estilos Arquiteturais.....	4
2.1.1. Arquitetura Monolítica.....	4
2.1.2. Microsserviços.....	4
2.1.3. Arquitetura em Camadas.....	4
2.1.4. Arquitetura Orientada a Eventos (EDA).....	4
2.1.5. Serverless e Cloud-Native.....	5
2.1.6. Model-View-ViewModel (MVVM).....	5
3. Escolha de Arquitetura.....	6
4. Tendências.....	7
4.1. Serverless e FaaS (Function as a Service).....	7
4.2. Arquiteturas Orientadas a Eventos (EDA).....	7
4.3. IA e Arquiteturas Orientadas por Dados (AI-driven).....	7
5. Conclusão.....	9
6. REFERÊNCIAS.....	10
7. GLOSSÁRIO.....	11

1. Conceitos

A Arquitetura de Software consiste na estruturação de uma aplicação, organizando suas funcionalidades, garantindo a escalabilidade e manutenção. De acordo com Agwenyi e Mbugua (2025), a arquitetura combina estilos arquiteturais e atributos de Qualidade com a estrutura de alto nível da abstração do sistema, operando através de processos de composição e decomposição de elementos. Diferente de outros Diagramas e Fluxogramas, ela não foca em direcionar a organização dos sistemas, mas sim em servir como orientador durante todo o processo de desenvolvimento, influenciando atividades críticas na criação e implementação do Software.

Uma visão proposta por Chippagiri (2025) define a construção de uma Arquitetura como um processo de tomada de decisão. Neste modelo, o arquiteto avalia diversas soluções alternativas para resolver um problema específico e seleciona aquela que atende de forma otimizada aos requisitos estabelecidos no levantamento. Assim, o escopo da arquitetura abrange desde a visão conceitual do sistema até as diretrizes técnicas que possibilitam sua execução e evolução.

A composição do conceito de Arquitetura de Software consiste na descrição de um conjunto de **Componentes**, suas **Relações** e as formas como eles **Interagem** e são **Implantados**. Para criar essas arquiteturas, geralmente são usados modelos como: a Modelagem Orientada a Objetos com UML (Unified Modeling Language), o Modelo de Visão 4+1 e as Linguagens de Descrição de Arquitetura (ADL).

Vale lembrar também que, a Arquitetura de Software também se preocupa muito com os Requisitos Não Funcionais (RNFs). Um design arquitetural robusto deve satisfazer não apenas as funcionalidades primárias, mas também garantir atributos como disponibilidade, portabilidade, confiabilidade e desempenho.

A Arquitetura depois de feita é vista como um conjunto de "decisões difíceis de mudar" porque suas escolhas fundamentais impactam todo o ciclo de vida do desenvolvimento de software (SDLC) e definem os limites do que o sistema pode ou não realizar no futuro. Decisões arquiteturais precoces, se inadequadas, podem levar à degradação do sistema e ao acúmulo de débito técnico. Por outro lado, uma arquitetura bem projetada reduz a carga cognitiva dos desenvolvedores, melhora a consistência do sistema e acelera os ciclos de entrega.

2. Padrões

Um padrão consiste numa solução padronizada e reutilizável para problemas recorrentes de design que surgem em contextos específicos. Segundo Chippagiri (2025), esses padrões operam como modelos que simplificam problemas complexos ao promoverem a modularidade, a escalabilidade e a manutenibilidade do programa.

2.1. Análise dos Principais Padrões e Estilos Arquiteturais

2.1.1. Arquitetura Monolítica

Historicamente dominante, o padrão de projeto monolítico estrutura a aplicação como uma única unidade lógica, sequencial e indivisível. Ela facilita o desenvolvimento inicial e os testes de integração em sistemas de pequena escala, porém encontra muita dificuldade de escalabilidade e baixa flexibilidade para atualizações.

2.1.2. Microsserviços

Este padrão envolve o design de aplicações como uma coleção de serviços fracamente acoplados e implantáveis de forma independente. Cada serviço é responsável por uma capacidade de negócio específica e comunica-se via APIs.

Possui vantagens como: agilidade, resiliência e a capacidade de usar diferentes tecnologias para serviços distintos.

2.1.3. Arquitetura em Camadas

Organiza o sistema em camadas distintas (ex: infraestrutura, plataforma e experiência do desenvolvedor), garantindo a separação de preocupações. Cada camada fornece serviços para a camada superior e consome da inferior.

Suas vantagens consistem em facilitar a manutenção e a clareza das responsabilidades do sistema.

2.1.4. Arquitetura Orientada a Eventos (EDA)

Permite que as aplicações respondem a mudanças de estado de forma assíncrona, desacoplando produtores de eventos de seus consumidores. Porém o

gerenciamento do fluxo de eventos e a depuração (*debugging*) tornam-se significativamente mais difíceis devido ao desacoplamento extremo.

Possuem vantagens de melhorar a responsividade e suportar aplicações de processamento em tempo real.

2.1.5. Serverless e Cloud-Native

Isola o gerenciamento de servidores, permitindo que os desenvolvedores foquem exclusivamente no código, enquanto o provedor de nuvem aloca recursos dinamicamente. Já o design *Cloud-Native* utiliza contêineres e microsserviços para garantir alta disponibilidade e escalabilidade através de ferramentas como o Kubernetes.

2.1.6. Model-View-ViewModel (MVVM)

É um padrão de interface gráfica que substitui o controlador por um ViewModel, utilizando mecanismos de data-binding.

O Model contém os dados, a View os elementos visuais e o ViewModel atua como uma abstração da visualização, facilitando testes unitários da lógica de apresentação.

A escolha de um padrão passa por uma análise rigorosa dos Requisitos Não Funcionais (RNFs). Agwenyi e Mbugua (2025) reforçam que uma arquitetura deve satisfazer requisitos de disponibilidade, escalabilidade, portabilidade e confiabilidade.

3. Escolha de Arquitetura

A escolha de uma arquitetura se baseia na necessidade de satisfazer as funcionalidades primárias quanto os Atributos de Qualidade (Requisitos Não Funcionais). Agwenyi e Mbugua (2025) destacam que critérios como disponibilidade, escalabilidade, portabilidade e confiabilidade são determinantes para a definição do plano diretor do sistema.

Outro fator de influência trata-se da fase de implantação, onde a transição de sistemas *on-premise* para arquiteturas Cloud-Native exigem que decisões considerem a compatibilidade e a flexibilidade de implantação.

A priorização de requisitos é essencial para definir a arquitetura adequada. Oladipo (2025) discute a tensão entre flexibilidade e padronização: a padronização excessiva pode gerar rigidez, enquanto a flexibilidade total pode levar à inconsistência e à duplicação de esforços. O processo de escolha, portanto, deve ser guiado por um "mindset" de produto, onde a arquitetura evolui continuamente com base no *feedback* dos usuários e nas realidades operacionais.

Uma decisão arquitetural precipitada pode acarretar consequências graves a longo prazo do projeto. Um dos riscos mais comuns é o desvio arquitetural, onde a implementação se desvia do design original, tornando o planejamento "inútil" e se caso for voltar atrás, muito difícil e demorada. Fuksa et al. (2025) alertam que desenvolvedores que selecionam implementações sem uma visão abrangente de alternativas correm o risco de introduzir complexidade desnecessária para GUIs simples ou sofrer com alto overhead de memória.

4. Tendências

A arquitetura de software passou por uma transformação radical na última década, deixando de ser vista como um artefato estático para tornar-se a espinha dorsal estrutural que governa a funcionalidade, escalabilidade e manutenibilidade dos sistemas. Nos últimos anos foi destacado o declínio das arquiteturas monolíticas, que dominaram o desenvolvimento por anos, em favor de microsserviços e modelos distribuídos. Essa evolução foi impulsionada por fatores críticos de mercado e tecnologia, principalmente com o surgimento de plataformas de nuvem como AWS, Azure e Google Cloud, que viabilizaram a adoção de containerização e orquestração. O movimento DevOps também desempenhou um papel central, integrando o desenvolvimento e as operações para acelerar a entrega de software.

As principais tendências em ascensão são:

4.1. Serverless e FaaS (Function as a Service)

O paradigma Serverless permite que desenvolvedores foquem exclusivamente no código, enquanto provedores de nuvem gerenciam a alocação dinâmica de recursos. Benefícios como o modelo de preço *pay-as-you-go* e a escalabilidade automática são contrabalançados por desafios como a latência de "cold start" e o aprisionamento tecnológico (*vendor lock-in*). Jonas et al. (2019) descrevem este modelo como uma simplificação essencial para a programação em nuvem.

4.2. Arquiteturas Orientadas a Eventos (EDA)

A EDA desacopla produtores e consumidores de eventos, permitindo que os sistemas reajam instantaneamente a mudanças de estado de forma assíncrona. Esse padrão é fundamental para aplicações de processamento em tempo real, embora introduza complexidade na visualização e depuração de fluxos de eventos.

4.3. IA e Arquiteturas Orientadas por Dados (AI-driven)

A integração de IA Generativa e LLMs está redefinindo o Ciclo de Vida da Arquitetura de Software (SALC), auxiliando desde o levantamento de requisitos até a reconstrução arquitetural a partir de códigos legados. Abrahão et al. (2025) sugerem que estamos entrando em uma era de Sistemas Humano-Cêntricos, onde a IA não apenas auxilia o arquiteto, mas permite a criação de interfaces adaptativas (AUIs) que aprendem com o comportamento do usuário. Além disso, a Computação de Borda (Edge Computing) surge para conectar a distribuição global à responsividade local em sistemas IoT.

A mudança para sistemas distribuídos trouxe desafios significativos. A complexidade da orquestração, a latência de rede e a manutenção da consistência de dados em microsserviços são problemas recorrentes.

O papel do Arquiteto de Software está sofrendo uma mutação profunda. De um designer de soluções técnicas, ele passa a atuar como um gestor de plataformas e um curador de decisões orientadas por IA.

5. Conclusão

Em resumo, a arquitetura de software consolidou-se como o alicerce indispensável para o desenvolvimento de sistemas modernos, evoluindo de modelos rígidos e centralizados para ecossistemas dinâmicos e distribuídos. A transição para tecnologias nativas da nuvem, microsserviços e computação sem servidor reflete a busca contínua por escalabilidade e resiliência, permitindo que as organizações respondam com agilidade às crescentes complexidades do mercado digital.

O futuro da disciplina aponta para uma integração cada vez maior com a inteligência artificial e um foco redobrado no equilíbrio entre a automação e as necessidades humanas. Construir uma arquitetura eficiente exige um processo constante de tomada de decisão e análise de compromissos técnicos, onde a escolha das ferramentas e padrões deve sempre priorizar a sustentabilidade e a facilidade de manutenção a longo prazo, garantindo que o software continue evoluindo de forma saudável e segura.

6. REFERÊNCIAS

ESPOSITO, Matteo et al. **Generative AI for Software Architecture: Applications, Challenges, and Future Directions**. Disponível em:

<https://arxiv.org/pdf/2503.13310>.

FUKSA, Mario; SPETH, Sandro; BECKER, Steffen. **MVVM Revisited: Exploring Design Variants of the Model-View-ViewModel Pattern**. Disponível em:

<https://arxiv.org/pdf/2504.18191>.

SU, Ruoyu et al. **Emerging Trends in Software Architecture from the Practitioner's Perspective: A Five-Year Review**. Disponível em:

<https://arxiv.org/pdf/2507.14554>

ABRAHÃO, Silvia et al. **Software Engineering by and for Humans in an AI Era**. Disponível em:

<https://www.ijisrt.com/trends-in-software-architecture-designs-evolution-and-current-state>

AGWENYI, C. A.; MBUGUA, S. M. **Trends in Software Architecture Designs: Evolution and Current State**. Disponível em:

<https://www.ijisrt.com/trends-in-software-architecture-designs-evolution-and-current-state>.

CHIPPAGIRI, Srinivas. **A Comprehensive Review of Software Design Patterns: Applications and Future Direction**. Disponível em:

<https://thercsas.com/wp-content/uploads/2025/02/rcsas5022025001.pdf>

OLADIPO, Adebayo T. **Architectural Patterns and Components of Internal Developer Platforms: Evidence from Multivocal Literature**. Disponível em:

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6709298

7. GLOSSÁRIO

API (*APPLICATION PROGRAMMING INTERFACE*) — Interface de Programação de Aplicações. Conjunto de rotinas, protocolos e ferramentas que estabelece as regras para que diferentes sistemas ou softwares possam se comunicar e trocar dados de forma padronizada;

DATA-BINDING — Vinculação de dados. Técnica de programação utilizada para sincronizar automaticamente os dados entre a interface do usuário (UI) e a lógica de negócios/modelo da aplicação, garantindo que alterações de um lado reflitam imediatamente no outro;

DEVOPS — Junção das palavras *Development* (Desenvolvimento) e *Operations* (Operações). Conjunto de práticas, culturas e ferramentas que integra as equipes de desenvolvimento de software e de infraestrutura, visando automatizar processos, acelerar entregas e aumentar a confiabilidade do sistema;

GUI (*GRAPHICAL USER INTERFACE*) — Interface Gráfica do Usuário. Tipo de interface visual composta por elementos como botões, janelas e ícones, que permite ao usuário interagir com o software por meio de dispositivos como mouse ou toque, em vez de comandos de texto;

KUBERNETES — Plataforma de código aberto para orquestração e automação do gerenciamento de contêineres de software, facilitando a implantação, o dimensionamento (*scaling*) e a operação de aplicações distribuídas;

LLM (*LARGE LANGUAGE MODEL*) — Modelo de Linguagem de Grande Porte. Algoritmo de Inteligência Artificial baseado em redes neurais profundas e treinado com grandes volumes de dados de texto, capaz de compreender, processar e gerar linguagem natural.

ON-PREMISE — Modelo de implantação de infraestrutura computacional no qual os softwares e servidores são instalados e mantidos localmente, nos próprios prédios ou instalações da organização, sob seu controle direto;

SDLC (*SOFTWARE DEVELOPMENT LIFE CYCLE*) — Ciclo de Vida de Desenvolvimento de Software. Estrutura conceitual que define as etapas seguidas no planejamento, criação, teste, implantação e manutenção de um software;

SERVERLESS — Computação Sem Servidor. Modelo de execução em nuvem no qual o provedor gerencia dinamicamente a alocação de servidores e recursos computacionais, cobrando apenas pelo tempo real de execução do código, desobrigando os desenvolvedores de gerenciar a infraestrutura;

UML (*UNIFIED MODELING LANGUAGE*) — Linguagem de Modelagem Unificada. Linguagem visual e padronizada utilizada para especificar, visualizar, construir e documentar a estrutura e os comportamentos de sistemas de software complexos;

VENDOR LOCK-IN — Dependência de Fornecedor. Situação na qual um cliente se torna excessivamente dependente dos produtos, serviços ou tecnologias de um único provedor, tornando o custo, o esforço ou a complexidade de migrar para uma solução concorrente proibitivos.